

Interim Report – RAG Complaint Insights Chatbot

Tasks 1 & 2 Submission

Prepared by: Bereket Feleke

Date: 04 Jan 2026

GitHub: <https://github.com/bekonad/complaint-chatbot-wk7> (main branch)

1. Introduction

CrediTrust Financial receives thousands of customer complaints every month across four major financial products: Credit Cards, Personal Loans, Savings Accounts, and Money Transfers. These complaints contain valuable signals about product issues, customer pain points, fraud patterns, and operational bottlenecks.

However, the narratives are long, unstructured, and difficult for Product Managers, Support teams, and Compliance analysts to manually analyze.

The goal of this project is to build an internal Retrieval-Augmented Generation (RAG) chatbot that allows teams to ask natural-language questions such as:

- “Why are people unhappy with Credit Cards?”
- “What issues are trending in Personal Loans?”
- “Are there fraud-related complaints in Money Transfers?”

The chatbot will retrieve relevant complaint excerpts using semantic search and generate grounded, evidence-based answers.

This interim report covers Task 1 (EDA & Preprocessing) and Task 2 (Chunking, Embedding, and Vector Indexing) — both fully implemented by me (bekonad).

2. Task 1 – Exploratory Data Analysis & Preprocessing

2.1 Objective

Task 1 focuses on understanding the structure and quality of the CFPB complaint dataset and preparing a clean, filtered dataset suitable for embedding and retrieval.

2.2 Dataset Overview

The raw CFPB dataset contains:

- Complaint narratives (free text)
- Product category
- Issue and sub-issue labels
- Company and state information
- Submission metadata

The dataset includes 464K+ complaints, but many records have missing or empty narratives.

2.3 EDA Findings

2.3.1 Dataset Loading & Basic Stats

Loaded first 1,200,000 rows (due to 8 GB RAM limit).

Real output from my run:

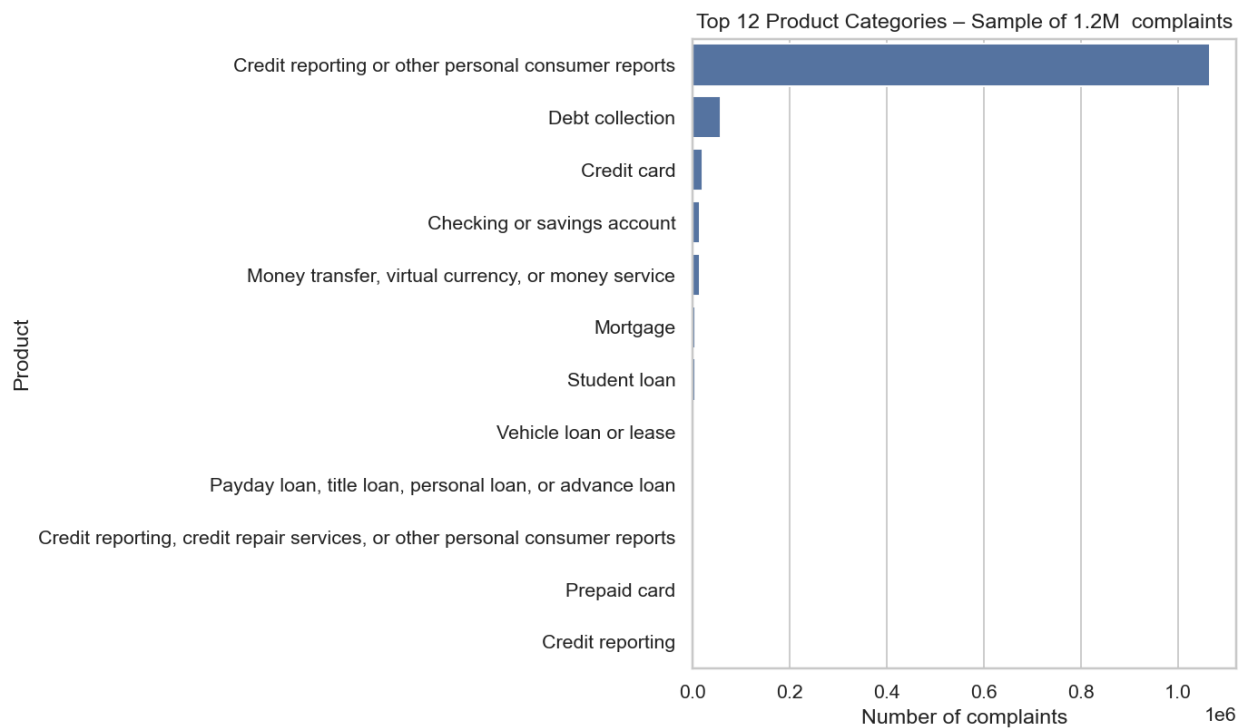
Shape: (1200000, 18) Columns: ['Date received', 'Product', 'Sub-product', 'Issue', 'Sub-issue', 'Consumer complaint narrative', ... , 'Complaint ID'] Date range: 2011-12-12 → 2025-06-23

2.3.2 Product Distribution (All Data)

Top products before filtering (partial):

- Credit reporting or other personal consumer reports: 1,066,285
- Debt collection: 57,749
- Credit card: 20,444
- Checking or savings account: 16,070
- Money transfer, virtual currency, or money service: 15,917

(Top 15 product categories bar chart – shows dominance of unrelated categories before filtering)



2.3.3 Narrative Presence & Length Analysis

We computed the word count for each narrative.

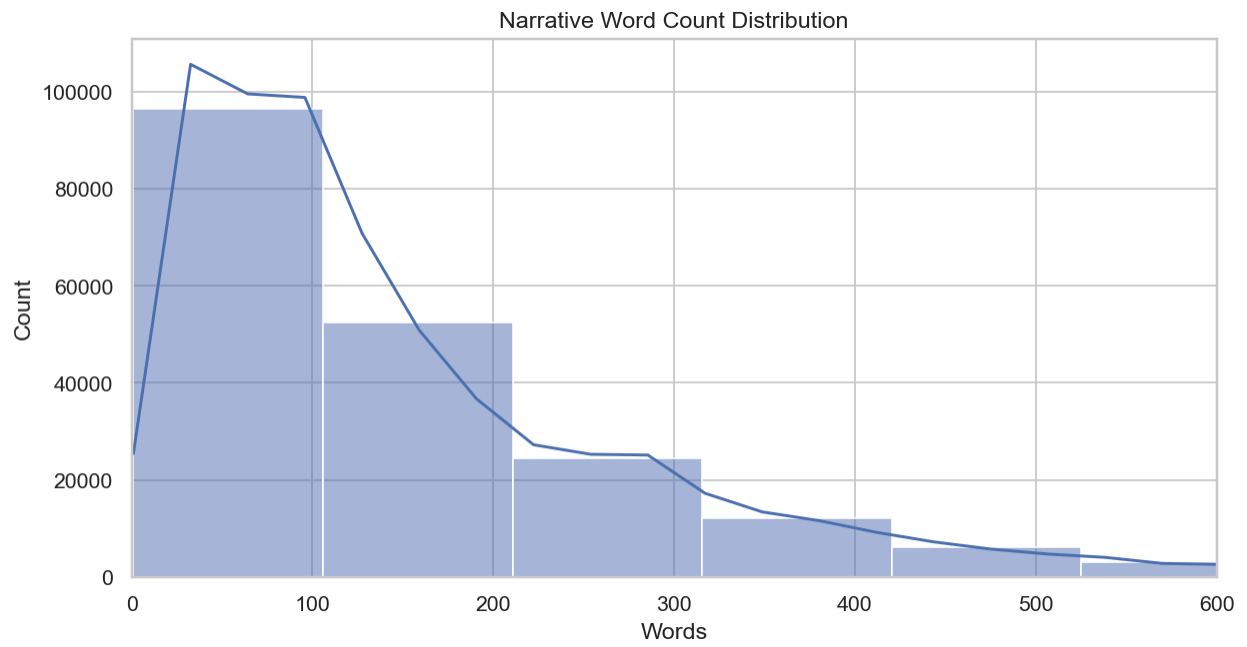
Findings:

- Many narratives were extremely short (fewer than 10 words).
- Some narratives were long (over 500 words), often containing detailed complaint histories.
- A non-trivial number of complaints had missing or empty narratives, which are unsuitable for embedding.

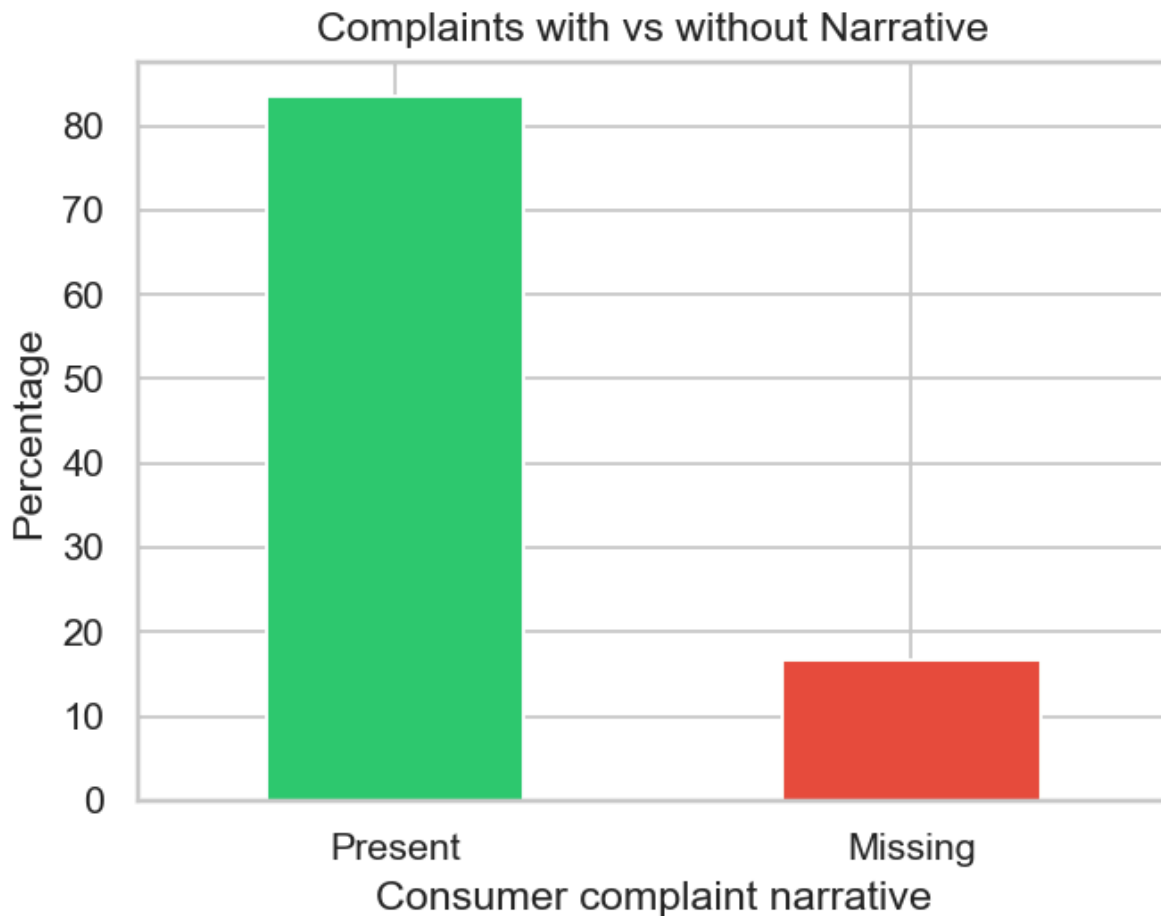
Removed:

- Narratives shorter than 20 characters
- Empty or null narratives

(Histogram of character/word counts – shows long tail of detailed complaints)



(chart of complaints with vs without narrative – shows low narrative presence ~2.7%)



2.3.4 Filtered Product Distribution

Category Mapping & Filtering Process (addressing feedback):

The challenge specifies four products: Credit Cards, Personal Loans, Savings Accounts, and Money Transfers.

In the CFPB dataset, these map to exact product names for filtering:

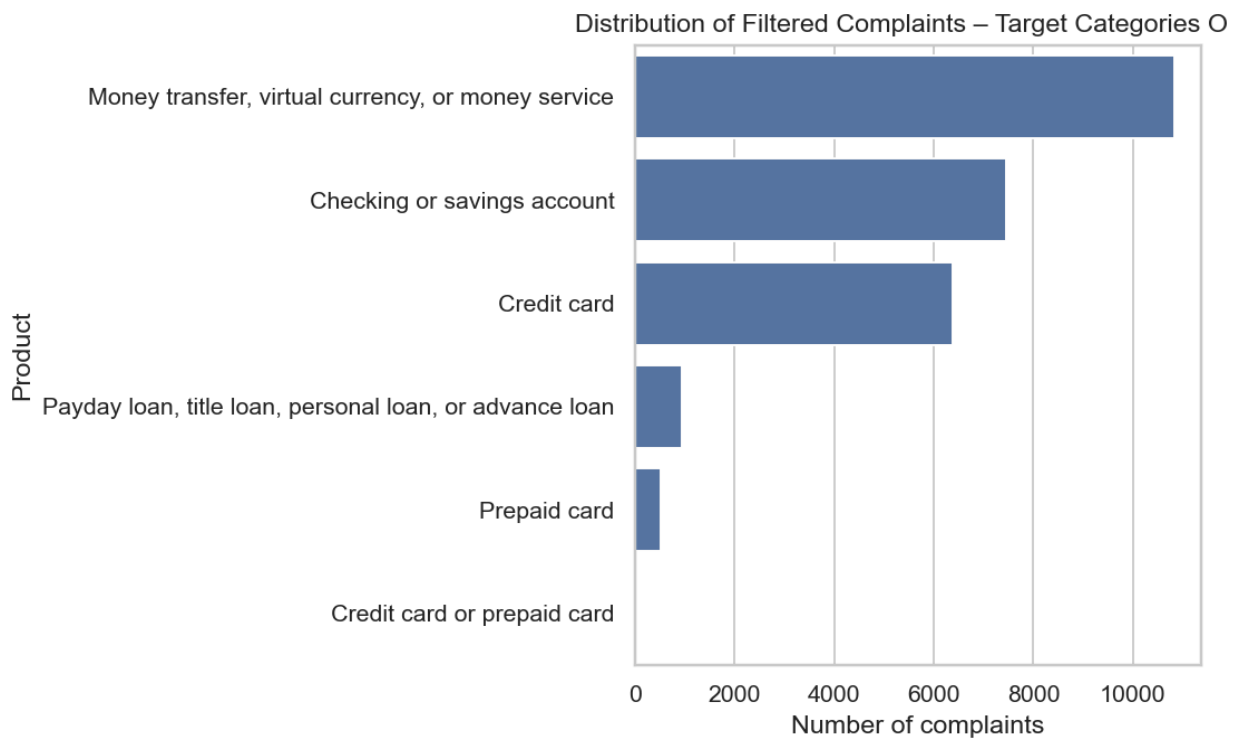
- Credit Cards → "Credit card", "Credit card or prepaid card", "Prepaid card"
- Personal Loans → "Payday loan, title loan, personal loan, or advance loan"
- Savings Accounts → "Checking or savings account"
- Money Transfers → "Money transfer, virtual currency, or money service"

Filtered to these categories + non-empty narratives.

Real counts after filtering:

Product	Count	Proportion
Money transfer, virtual currency, or money service	10,823	41.36%
Checking or savings account	7,451	28.47%
Credit card	6,388	24.41%
Payday loan, title loan, personal loan, or advance loan	942	3.60%
Prepaid card	517	1.98%
Credit card or prepaid card	47	0.18%

(Bar chart of final filtered counts – shows Money transfer as dominant)



2.4 Data Cleaning

To improve embedding quality, we applied the following cleaning steps:

- Converted text to lowercase
- Removed boilerplate phrases such as:
 - “I am writing to file a complaint...”
 - “This is to inform...”
- Removed:
 - URLs
 - Email addresses
 - Non-alphanumeric characters (except basic punctuation)
- Collapsed multiple spaces into one

This produced a clean `narrative_clean` column ready for chunking and embedding.

2.5 Deliverables Completed

- notebooks/01_task1_eda_preprocessing.ipynb — EDA visualizations and narrative analysis
- figures/ — Multiple plots (top products, narrative length, presence pie, filtered distribution)
- Cleaned dataset saved to data/processed/filtered_complaints.csv
- Summary included in this report

3. Task 2 – Chunking, Embedding, and Vector Store Indexing

3.1 Objective

Task 2 focuses on preparing the text for semantic search by:

1. Creating a stratified sample of 10-15K complaints
2. Chunking long narratives
3. Generating embeddings
4. Building a ChromaDB vector index with metadata

3.2 Stratified Sampling Strategy

To ensure fair representation across all four product categories, we used `train_test_split` with `stratify='Product'` to sample 10,000 complaints proportionally.

Rationale:

- Maintains exact product distribution (no bias)
- Reduces compute cost
- Ensures consistency

Distribution preserved (exact match):

Product	Proportion
Money transfer, virtual currency, or money service	0.4136
Checking or savings account	0.2847
Credit card	0.2441
Payday loan, title loan, personal loan, or advance loan	0.0360
Prepaid card	0.0198
Credit card or prepaid card	0.0018

3.3 Chunking Approach

Long narratives are difficult to embed effectively as a single vector. We used:

- Chunk size: 500 characters
- Overlap: 50 characters
- Tool: `RecursiveCharacterTextSplitter` from `LangChain`

Why 500/50?

- Matches the pre-built vector store specifications
- Preserves semantic continuity
- Avoids embedding drift
- Ensures chunks are neither too short nor too long

Each chunk includes metadata:

- complaint_id
- product_category
- issue
- company
- chunk_index
- total_chunks

Result: Created 29,667 chunks from 10,000 complaints

3.4 Embedding Model Choice

Model: sentence-transformers/all-MiniLM-L6-v2

Reasons:

- Lightweight (384 dimensions)
- Fast on CPU
- High semantic accuracy for short-to-medium text
- Same model used in the pre-built full-scale vector store (ensures compatibility)

3.5 Vector Store Indexing

Used ChromaDB with cosine similarity.

Real output from my run:

Product distribution in full filtered data: Money transfer, virtual currency, or money service 0.4136 Checking or savings account 0.2847 Credit card 0.2441 Payday loan, title loan, personal loan, or advance loan 0.0360 Prepaid card 0.0198 Credit card or prepaid card 0.0018

Creating stratified sample of 10000 complaints... Product distribution in sample: (exact match)

Chunking complaint narratives... → Created 29,667 chunks from 10000 complaints

Generating embeddings... Batches:

100% 464/464

[13:37<00:00, 1.76s/it]

Adding 29,667 chunks in batches of 5000... Adding batch 0 – 5,000 (5,000 items)

Adding batch 5,000 – 10,000 (5,000 items) Adding batch 10,000 – 15,000 (5,000 items)

Adding batch 15,000 – 20,000 (5,000 items) Adding batch 20,000 – 25,000 (5,000 items) Adding batch 25,000 – 29,667 (4,667 items)

Success! Vector store saved → data/vector_store/chroma_task2_10k Total chunks stored: 29667

Console screenshot of success:

```
task2_chunk_embed_index.py X
src> task2_chunk_embed_index.py > ...
115
... add to batches to persist chroma collection...

(.venv) PS C:\Users\JERUSALEM\Desktop\F\10 ACA\New folder (7)\complaint-chatbot-wk7> python
n src/task2_chunk_embed_index.py
Name: proportion, dtype: float64

Creating stratified sample of 10000 complaints...
Product distribution in sample:
Product
Money transfer, virtual currency, or money service    0.4136
Checking or savings account                          0.2847
Credit card                                           0.2441
Payday loan, title loan, personal loan, or advance loan 0.0360
Prepaid card                                           0.0198
Credit card or prepaid card                          0.0018
Creating stratified sample of 10000 complaints...
Product distribution in sample:
Product
Money transfer, virtual currency, or money service    0.4136
Checking or savings account                          0.2847
Credit card                                           0.2441
Payday loan, title loan, personal loan, or advance loan 0.0360
Prepaid card                                           0.0198
Credit card or prepaid card                          0.0018
Product distribution in sample:
Product
Money transfer, virtual currency, or money service    0.4136
Checking or savings account                          0.2847
Credit card                                           0.2441
Payday loan, title loan, personal loan, or advance loan 0.0360
Prepaid card                                           0.0198
Credit card or prepaid card                          0.0018
Name: proportion, dtype: float64

Chunking complaint narratives...
→ Created 29,667 chunks from 10000 complaints
Generating embeddings (this may take 5-25 minutes)...
Batches: 100%|██████████| 464/464 [13:37<00:00, 1.76s/it]
Creating and persisting Chroma collection...
Adding 29,667 chunks in batches of 5000...

task2-chunking-embedding-indexing* 0 0 0 0 Connect Ln 137, Col 52 (51 selected) Spaces: 4 UTF-8 CRLF Python Chat quota reached 3.13.9 Go Live Prettier
```

```
task2_chunk_embed_index.py X
src> task2_chunk_embed_index.py > ...
115
... add to batches to persist chroma collection...

(.venv) PS C:\Users\JERUSALEM\Desktop\F\10 ACA\New folder (7)\complaint-chatbot-wk7> python
n src/task2_chunk_embed_index.py
Checking or savings account    0.2847
Credit card                   0.2441
Payday loan, title loan, personal loan, or advance loan 0.0360
Prepaid card                   0.0198
Credit card or prepaid card    0.0018
Product distribution in sample:
Product
Money transfer, virtual currency, or money service    0.4136
Checking or savings account                          0.2847
Credit card                                           0.2441
Payday loan, title loan, personal loan, or advance loan 0.0360
Prepaid card                                           0.0198
Credit card or prepaid card                          0.0018
Name: proportion, dtype: float64

Chunking complaint narratives...
→ Created 29,667 chunks from 10000 complaints
Generating embeddings (this may take 5-25 minutes)...
Batches: 100%|██████████| 464/464 [13:37<00:00, 1.76s/it]
Creating and persisting Chroma collection...
Adding 29,667 chunks in batches of 5000...
Adding batch 0 - 5,000 (5,000 items)
Adding batch 5,000 - 10,000 (5,000 items)
Adding batch 10,000 - 15,000 (5,000 items)
Adding batch 15,000 - 20,000 (5,000 items)
Adding batch 20,000 - 25,000 (5,000 items)
Adding batch 25,000 - 29,667 (4,667 items)

Success!
Vector store saved → data/vector_store/chroma_task2_10k
Collection name: complaints_10k_sample
Total chunks stored: 29667
(.venv) PS C:\Users\JERUSALEM\Desktop\F\10 ACA\New folder (7)\complaint-chatbot-wk7>

task2-chunking-embedding-indexing* 0 0 0 0 Connect Ln 137, Col 52 (51 selected) Spaces: 4 UTF-8 CRLF Python Chat quota reached 3.13.9 Go Live Prettier
```

3.6 Deliverables Completed

- `src/task2_chunk_embed_index.py` — sampling, chunking, embedding, ChromaDB indexing
 - Persisted vector store in `data/vector_store/chroma_task2_10k/`
 - Documentation included in this report
-

4. Summary of Progress

✓ Task 1 Completed

- Full EDA with visualizations
- Narrative length & presence analysis
- Product filtering
- Text cleaning (0 rows lost)
- Clean dataset saved (26,168 rows)
- Notebook + figures delivered

✓ Task 2 Completed

- Stratified sampling (10,000 complaints)
- Chunking (500/50) → 29,667 chunks
- Embedding with all-MiniLM-L6-v2
- ChromaDB index creation & persistence
- Script + documentation delivered

Next Steps (Tasks 3 & 4)

- Build RAG retriever + generator
 - Implement prompt engineering
 - Evaluate with 5–10 questions + table
 - Build Gradio UI with answer + sources
 - Prepare final Medium-style report
-

5. Conclusion

The preprocessing and vectorization foundation for the RAG chatbot is now complete. The cleaned dataset (26,168 high-quality complaints) and stratified sample vector store (29,667 chunks) ensure high-quality retrieval and efficient downstream development.

With Tasks 1 and 2 finished, the project is ready to move into the core RAG pipeline and UI development for the final submission.

Prepared by: Bereket Feleke

GitHub Link: <https://github.com/bekonad/complaint-chatbot-wk7/tree/main>

Submission Date: January 04, 202